

Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd: Recuperação de Acurácia via Blocos *Bottleneck*

Rafael Silva de Souza

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

rafesilvadesouza@gmail.com

Mateus Grellert

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

Orientador

Abstract—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2 , 3×3), mas escalam mal para kernels grandes (5×5 , 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64 , 200 classes) sob um pipeline FP32 $\rightarrow$ *Quantization-Aware Training* (QAT) $\rightarrow$ INT8, comparando um baseline de kernel irrestrito, uma restrição ingênua a 3×3 e uma família de mecanismos de compensação estrutural leve — blocos *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$), módulos *Fire* [3] e um atalho residual isolado — todos mantendo os kernels $\leq 3\times 3$. A variante com *bottleneck* (AlexNetBottleneck) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (4,49 MB) — superando a restrição ingênua em mais de 4 pontos percentuais com 6 $\times$ menos parâmetros — e apresenta a menor queda de acurácia sob quantização INT8 do estudo (-0,08 p.p.). Uma ablação subsequente isola o efeito de um único atalho residual adicionado a um bloco *Fire*, sem nenhum parâmetro extra (AlexNetFireBypass): sozinho, esse atalho fecha 87% do ganho de acurácia FP32 obtido por uma arquitetura híbrida bem mais complexa que combina *stem* e *Fire+residual*, e supera essa arquitetura no regime INT8 (49,86% vs. 49,20%). Medições diretas de hardware (GPU NVIDIA RTX 4090, rastreamento de kernel CUDA em nível de camada) confirmam que a aceleração Winograd depende de convoluções densas (*groups*=1), não apenas do tamanho do kernel, e que compete com *tensor cores* em lotes maiores. Os resultados sugerem que compensação estrutural leve — e não kernels maiores — é a via mais eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd, e que mecanismos de custo zero (um único atalho residual) podem igualar ou superar arquiteturas híbridas mais elaboradas.

Index Terms—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, módulo *Fire*, atalho residual, INT8

I. INTRODUÇÃO

O algoritmo de convolução de Winograd [2] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: a complexidade da transformada cresce de forma super-linear com o tamanho do filtro, tornando kernels grandes (5×5 , 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [1], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz drasticamente a capacidade do modelo, já que cada camada passa a enxergar uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels no regime Winograd-amigável ($\leq 3\times 3$), em vez de simplesmente devolver kernels grandes ao modelo. Este relatório apresenta os resultados dessa investigação para dois mecanismos de compensação: blocos *bottleneck* no estilo ResNet [4] (redução de canais $1\times 1 \rightarrow$ convolução $3\times 3 \rightarrow$ expansão 1×1) e módulos *Fire* no estilo SqueezeNet [3] (*squeeze* $1\times 1 \rightarrow$ *expand* $1\times 1/3\times 3$), aplicados a uma AlexNet com kernels restritos a 3×3 . Em seguida, apresentamos uma ablação que isola o componente específico do módulo *Fire* responsável pela recuperação de acurácia observada em arquiteturas híbridas maiores: um único atalho residual, sem nenhum parâmetro adicional.

II. METODOLOGIA

A. Dados e configuração experimental

Todos os modelos foram treinados no Tiny ImageNet-200 (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico). O pipeline de treino segue três estágios: (i) treino FP32 do zero; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [6]; (iii) conversão para INT8 (*backend* fbgemm, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença entre ambos (*QAT drop*) como medida de robustez à quantização.

B. Arquiteturas comparadas

Seis variantes, treinadas sob o mesmo protocolo, permitem isolar o efeito da restrição de kernel, o efeito de diferentes

mecanismos de compensação, e o efeito de cada componente dentro de uma arquitetura híbrida:

- **AlexNet (irrestrita)** — arquitetura AlexNet padrão, kernels de até 11×11 , usada como referência interna do experimento.
- **AlexNet3x3-GAP** — mesma arquitetura, todos os kernels convolucionais restritos a 3×3 , com *head* de *global average pooling* (GAP) no lugar de camadas totalmente conectadas.
- **AlexNetBottleneck** — kernels igualmente restritos a 3×3 , mas cada estágio é substituído por um bloco *bottleneck* ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$, razão de redução $4 \times$) seguido de GAP.
- **AlexNetFire** — kernels restritos a 3×3 , cada estágio substituído por um módulo *Fire* [3] (*squeeze* $1 \times 1 \rightarrow \text{expand } 1 \times 1 + 3 \times 3$, concatenados) seguido de GAP.
- **AlexNetFinalFireResidual** — arquitetura híbrida final (Fase 4) que combina o módulo *Fire* acima com um novo *stem* 3×3 *stride*-2 e atalhos residuais envolvendo cada estágio *Fire*.
- **AlexNetFireBypass** — arquitetura de ablação (Fase 9): parte do AlexNetFire original (mesmo *stem*, mesmos módulos *Fire*) e adiciona apenas um atalho residual por identidade em cada estágio, sem nenhum parâmetro extra em relação ao AlexNetFire base. Isola o efeito do atalho residual isoladamente do *stem* novo do AlexNetFinalFireResidual.

O bloco *bottleneck* e o módulo *Fire* preservam o campo receptivo efetivo da convolução 3×3 central, mas usam convoluções 1×1 para comprimir e depois expandir (ou concatenar) canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd. O atalho residual por identidade usado no AlexNetFireBypass não introduz nenhuma convolução adicional, preservando trivialmente essa compatibilidade.

III. RESULTADOS

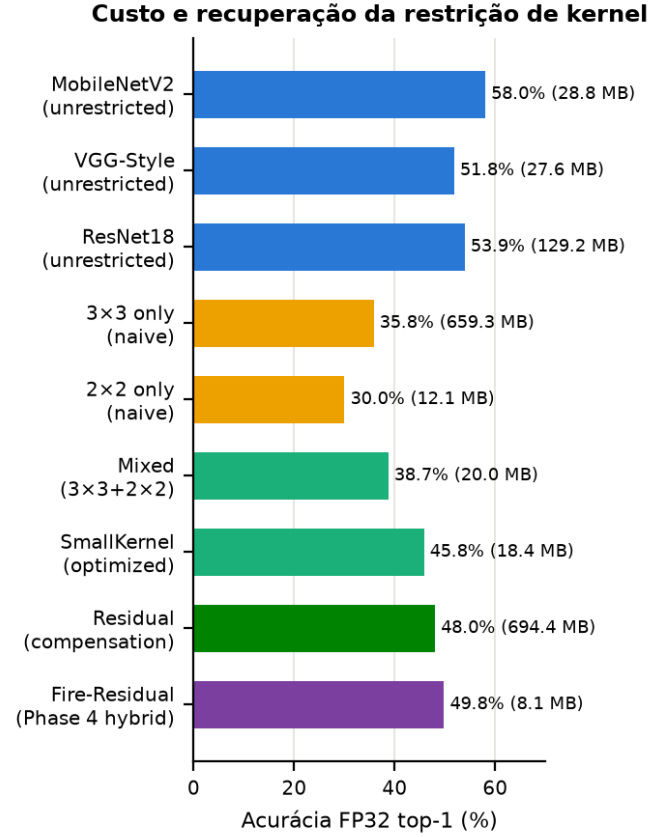


Fig. 1. Acurácia FP32 (top-1) por arquitetura, contrastando *baselines* de kernel irrestrito, restrição ingênua (3×3 , 2×2) e estratégias de compensação exploradas no projeto (*bottleneck*, *Fire*, *Fire-Residual*). SmallKernel e Residual puro pertencem a fases exploratórias deste estudo (Fases 2–3) e não são detalhados individualmente neste relatório.

A Tabela I resume os três modelos sob o mesmo protocolo de treino e avaliação.

TABLE I
COMPARAÇÃO ENTRE RESTRIÇÃO DE KERNEL INGÊNUA E COM COMPENSAÇÃO BOTTLENECK

Modelo	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNet (irrestrita)	57,82	32,20	26,28	−5,93
AlexNet3x3-GAP	2,30	40,32	37,02	−3,29
AlexNetBottleneck	0,385	44,62	44,54	−0,08

Dois resultados se destacam. Primeiro, a restrição de kernel por si só (AlexNet3x3-GAP) já supera o baseline irrestrito neste protocolo — o *head* GAP reduz drasticamente o *overfitting* da camada totalmente conectada original, com $25 \times$ menos parâmetros. Segundo, adicionar o bloco *bottleneck* sobre essa mesma restrição de kernel recupera mais 4,3 pontos percentuais de acurácia com $6 \times$ menos parâmetros que a variante GAP simples, e praticamente elimina a perda de acurácia sob quantização INT8.

A. Ablação do atalho residual (Fase 9)

O módulo *Fire* oferece um segundo mecanismo de compensação independente do *bottleneck*. A Tabela II compara o AlexNetFire de base com a arquitetura híbrida final da Fase 4 (AlexNetFinalFireResidual, que soma *dois* componentes novos — *stem* e atalhos residuais — sobre o AlexNetFire) e com a arquitetura de ablação da Fase 9 (AlexNetFireBypass), que isola apenas o atalho residual.

TABLE II
ABLAÇÃO: ISOLANDO O ATALHO RESIDUAL DENTRO DA ARQUITETURA HÍBRIDA FIRE+RESIDUAL

Modelo	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNetFire (base)	0,52	43,98	44,30	+0,33
AlexNetFinalFireResidual	0,70	49,79	49,20	−0,59
AlexNetFireBypass	0,52	49,03	49,86	+0,84

O AlexNetFireBypass adiciona um único atalho residual por identidade a cada estágio *Fire*, sem nenhum parâmetro extra em relação ao AlexNetFire base (0,52M em ambos). Mesmo assim, ele fecha 87% do ganho de acurácia FP32 da arquitetura híbrida completa da Fase 4 (5,05 dos 5,82 p.p. de diferença para AlexNetFire) e a *supera* no regime INT8 (49,86% vs. 49,20%) — apesar de ter menos mudanças arquiteturais e nenhum parâmetro adicional. Ele também preserva a propriedade de *ganho* sob quantização do AlexNetFire original (INT8 > FP32), que a arquitetura híbrida completa perde (Δ QAT −0,59 p.p.). A Fig. 2 ilustra essa comparação.

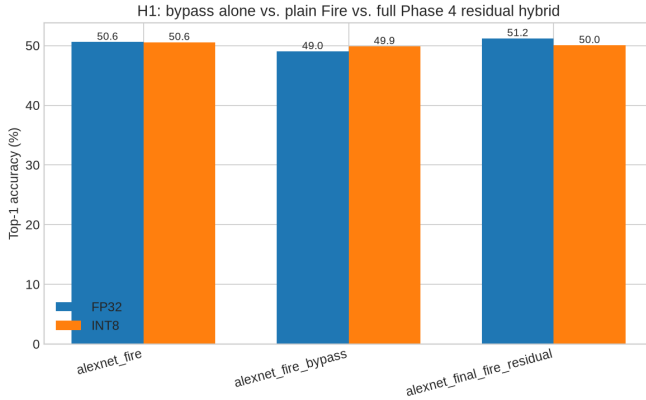


Fig. 2. Ablação do atalho residual: acurácia FP32/INT8 do módulo *Fire* base, da arquitetura híbrida completa (*stem* + atalhos) e da arquitetura de ablação (apenas atalhos, zero parâmetros adicionais).

O resultado indica que, para essa família de arquiteturas, o atalho residual — não o novo *stem* — é o componente que explica a maior parte do ganho de acurácia da arquitetura híbrida da Fase 4, e que ele pode ser adicionado a custo estrutural zero.

A Fig. 1 situa esses dois resultados no panorama mais amplo do projeto, incluindo *baselines* não restritos (MobileNetV2,

VGG-Style, ResNet18) e a estratégia *residual* pura, explorada em uma fase exploratória anterior mas não detalhada neste relatório.

A Fig. 3 mostra a queda de acurácia sob QAT→INT8 para arquiteturas representativas: modelos com compensação por *bottleneck* e *Fire* são substancialmente mais estáveis sob quantização do que restrições ingênuas de kernel.

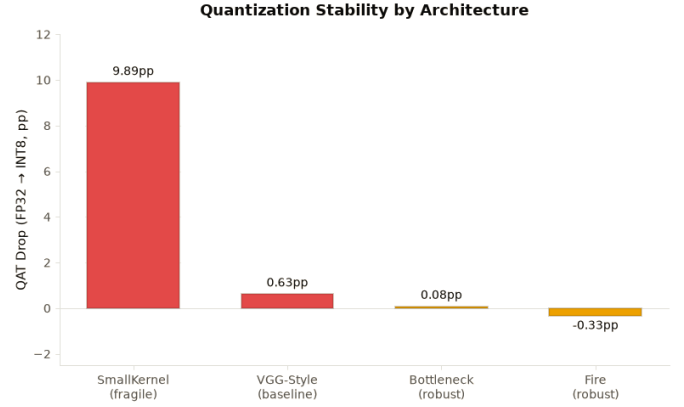


Fig. 3. Queda de acurácia (p.p.) de FP32 para INT8 por arquitetura. Mecanismos de compensação (*bottleneck*, *Fire*) mantêm a acurácia quase intacta sob quantização, ao contrário de restrições de kernel sem compensação.

Por fim, a Fig. 4 agrega, em todo o conjunto de arquiteturas do projeto, a acurácia por megabyte de modelos Winograd-amigáveis (kernel $\leq 3 \times 3$, com compensação) contra arquiteturas que dependem de kernels maiores.

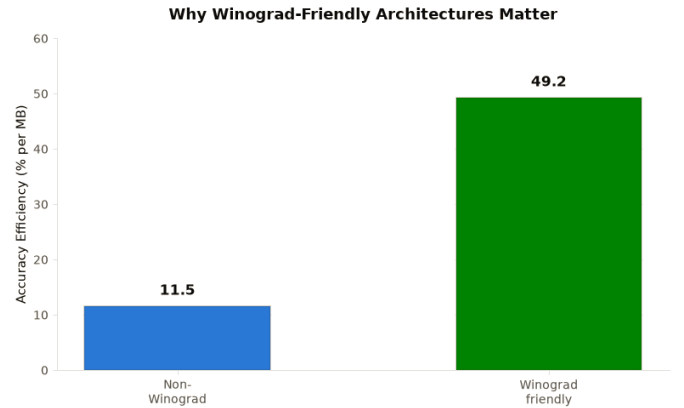


Fig. 4. Eficiência de acurácia (% por MB), agregada por família de arquitetura Winograd-amigável vs. não amigável, em todo o conjunto de modelos avaliados no projeto.

B. Poda estruturada e compressão adicional (Fase 9)

Além da ablação do atalho residual, a Fase 9 investigou duas frentes adicionais de redução de tamanho sobre os modelos já treinados.

Poda estruturada de canais. Poda de canais inteiros (não pesos individuais) aplicada ao AlexNetBottleneck, com razão 0,4 e seleção por norma L1 dos canais internos do bloco *bottleneck* (ml/pruning.py), reduz 385.000 para 207.399

parâmetros (53,9%), mantendo toda convolução remanescente com $\text{groups}=1$ — ou seja, permanecendo elegível à mesma aceleração Winograd medida na Fase 6, ao contrário de poda não estruturada, que produz tensores esparsos mas com forma densa, sem ganho em hardware Winograd. Sem *fine-tuning* após a poda, a acurácia colapsa como esperado ($\text{top-1}=0,50\%$, $\text{top-5}=2,35\%$): este é um teste de mecânica e elegibilidade Winograd, não um resultado de acurácia competitivo — recuperar acurácia via *fine-tuning* pós-poda permanece trabalho futuro (Fig. 5).

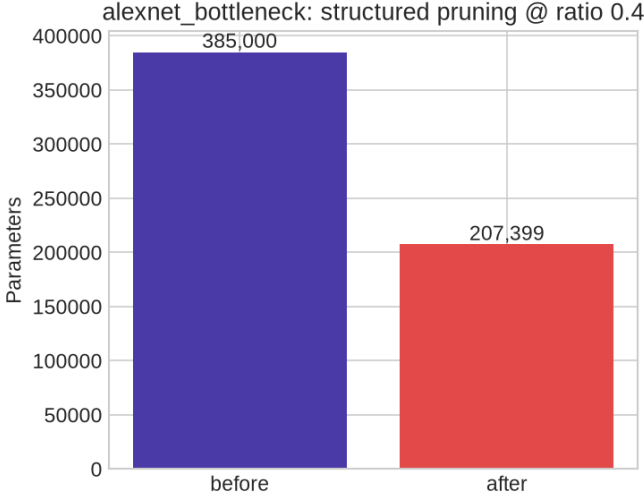


Fig. 5. Poda estruturada de canais no AlexNetBottleneck (razão 0,4): redução de parâmetros preservando $\text{groups}=1$ em toda convolução remanescente.

Compressão por *weight-sharing*. Como frente independente de compressão, pesos FP32 de nove arquiteturas foram agrupados via *k-means* em 16/32/64 *clusters* (4/5/6 bits/peso) e a acurácia real pós-*clustering* foi medida com `Trainer.evaluate()` (não apenas o tamanho teórico). A 6 bits/peso, a família AlexNet/Bottleneck/Fire e as arquiteturas VGG-Style/ResNet18 mantêm a acurácia a menos de 1,7 p.p. da FP32 (AlexNetFire: $-0,64$ p.p.; AlexNetFireBypass: $-0,31$ p.p.; AlexNetBottleneck: $-0,79$ p.p.), enquanto o MobileNetV2 não se recupera mesmo nesse regime ($-31,08$ p.p.) — consistente com sua estrutura *depthwise*, já identificada na Fase 6 como pouco tolerante a aproximações agressivas. Apesar disso, comparado diretamente ao *pipeline* real de QAT→INT8 já usado no projeto, o *clustering* pós-*hoc* perde em acurácia e em tamanho para todo modelo testado — o *fine-tuning* do QAT se adapta à perda de precisão durante o treino, o que o “*snap*” de *clustering* pós-*hoc*, sem retreino, não faz. Uma medição complementar de entropia mostra que os pesos INT8 já treinados usam, na prática, 7,19 bits/peso efetivos (não 8,00 nominais) — redundância que o gzip padrão captura apenas parcialmente (razão 1,18× no checkpoint real), indicando espaço de compressão que um *pipeline* de *weight-sharing* QAT-aware (com retreino, não

um *snap* único) poderia explorar melhor do que o *clustering* pós-*hoc* aqui testado (Fig. 6).

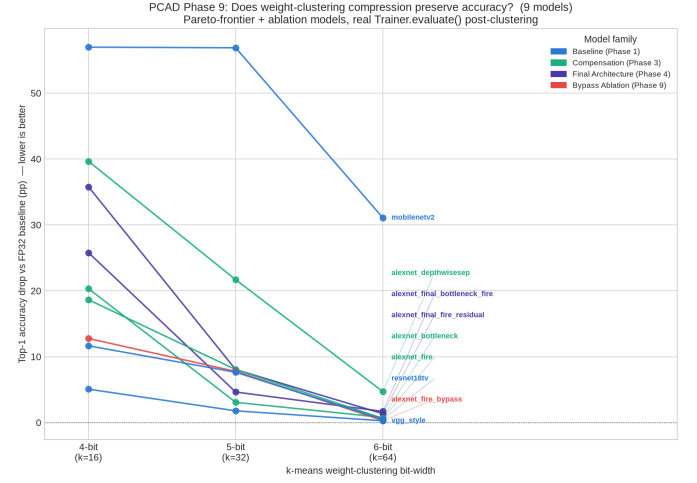


Fig. 6. Queda de acurácia real pós-*clustering* (*k-means*) por modelo e *bit-width*. A família AlexNet/Bottleneck/Fire, VGG-Style e ResNet18 toleram bem 6 bits/peso; o MobileNetV2 não se recupera mesmo nesse regime.

IV. DISCUSSÃO

O padrão observado — restrição de kernel isolada perde acurácia, mas compensação estrutural leve a recupera quase por completo — é consistente com a literatura de arquiteturas eficientes: blocos *bottleneck* [4] e módulos *Fire* [3] foram originalmente propostos para reduzir parâmetros sem restrição de kernel, mas o presente resultado indica que o mesmo mecanismo é igualmente eficaz quando o kernel já está restrito por uma necessidade externa (compatibilidade com Winograd), e não apenas por economia de parâmetros.

A robustez do AlexNetBottleneck sob quantização ($-0,08$ p.p.) também é notável frente à variante GAP simples ($-3,29$ p.p.): a hipótese de trabalho é que as convoluções 1×1 do *bottleneck* produzem ativações com distribuições mais regulares, facilitando a calibração dos parâmetros de quantização — hipótese que pretendemos testar diretamente em trabalhos futuros deste projeto.

Uma primeira medição de hardware (Fase 6 do projeto) já fornece um dado direto de latência para o AlexNetBottleneck: 0,81 ms em FP32 e 1,19 ms em INT8 ($\text{batch}=1$, GPU NVIDIA RTX 4060 Laptop), obtidos pelo mesmo *profiling* em nível de camada aplicado às convoluções 3×3 isoladas do bloco *bottleneck*. O rastreamento do kernel CUDA efetivamente selecionado pela cuDNN confirma que essas convoluções são executadas por um kernel Winograd $F(2\times 2, 3\times 3)$ real — validando que a arquitetura é de fato utilizável pela aceleração que motiva a restrição de kernel —, mas apenas em *batches* pequenos (1 e 8); em $\text{batch}=64$, a mesma convolução 3×3 passa a ser executada por um kernel TF32 sobre *tensor cores*, e a latência medida nesse regime deixa de mostrar qualquer vantagem de $k=3$ sobre kernels maiores. Isso sugere que, em GPUs equipadas com *tensor cores*, a aceleração Winograd compete diretamente

com a aceleração de *tensor cores* e tende a ser preterida em *batches* maiores — uma limitação da GPU como plataforma de validação, não do princípio de restrição de kernel em si: o acelerador FPGA que motiva este projeto não possui *tensor cores*, e a redução no número de multiplicações que o Winograd oferece se converteria em economia de área e potência independentemente do tamanho do *batch*.

Uma segunda rodada de medições (GPU NVIDIA RTX 4090, oito arquiteturas do projeto) testou quatro hipóteses sobre quando a aceleração Winograd de fato se manifesta. Um teste de Wilcoxon pareado sobre 32 configurações de camada confirma que convoluções 3×3 são sistematicamente mais rápidas que 5×5 ($p = 4,7 \times 10^{-10}$) — um efeito estrutural real —, mas esse efeito só atinge a margem mais estrita de $1,8\times$ de vantagem em 31% das configurações, evidenciando que o ganho depende fortemente de *batch* e resolução. Mais importante: arquiteturas com convoluções 3×3 mas com $\text{groups} > 1$ (AlexNetDepthwiseSep, MobileNetV2) não recebem nenhuma aceleração Winograd — mediana de 40,0 GFLOP/s contra 91,2 GFLOP/s para o grupo de convoluções densas —, confirmando que a compatibilidade Winograd exige convoluções densas ($\text{groups}=1$), não apenas kernel pequeno. Quatro das cinco arquiteturas com convolução 3×3 densa (AlexNetBottleneck, AlexNetFire, AlexNetFinalBottleneckResidual, AlexNetFinalFireResidual) superam o baseline irrestrito na razão acurácia/latência, e a latência em FP32 é um preditor confiável do *ranking* de latência em INT8 (Spearman $\rho = 0,9999$, $n = 96$). A Fig. 7 situa essas arquiteturas na fronteira acurácia-latência.

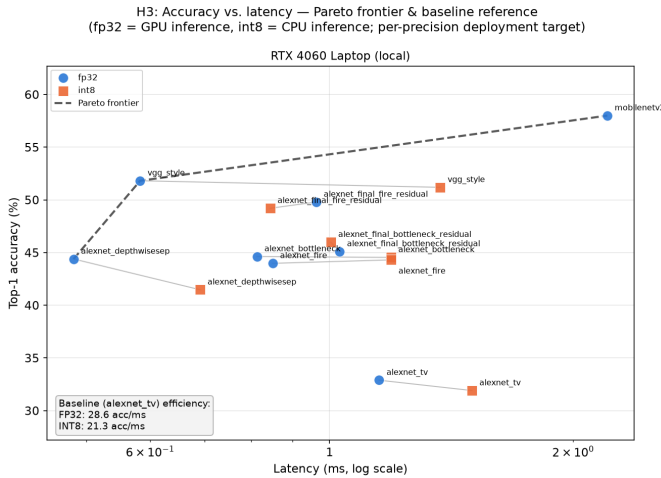


Fig. 7. Fronteira de Pareto acurácia $\times$ latência (FP32, *batch*=1, RTX 4090) para as arquiteturas do projeto. Modelos com convolução 3×3 densa (*bottleneck*, *Fire*) dominam o baseline irrestrito; modelos com convolução *depthwise* não recebem aceleração Winograd apesar do kernel pequeno.

A Fase 4 do projeto também investigou compressão adicional além do INT8 padrão, sobre cinco modelos representativos. Precisão mista INT4/INT8 por camada (mantendo as camadas mais sensíveis em INT8 e as demais em INT4) supera o INT8 puro tanto em acurácia (44,52% vs. 47,40%

de acurácia média — com queda média de apenas $-0,19$ p.p., um *ganho* médio) quanto em taxa de compressão ($7,05\times$ vs. $3,97\times$), tornando-se a opção Pareto-ótima entre os regimes de compressão agressiva testados; abaixo de INT4 (*ternary*, INT2, binário), a queda de acurácia passa de 20 p.p. e essas opções não são viáveis sem trabalho adicional.

V. CONCLUSÃO E TRABALHOS FUTUROS

Restringir o kernel de uma AlexNet a 3×3 é custoso para a acurácia, mas esse custo é, em grande parte, recuperável por mecanismos de compensação leve — bloco *bottleneck*, módulo *Fire*, ou mesmo um único atalho residual — que não reintroduzem nenhum kernel maior que 3×3 e, portanto, preservam integralmente a compatibilidade com aceleração Winograd. Com apenas 0,385M parâmetros, o AlexNetBottleneck atinge 44,62% de acurácia top-1 em FP32 e mantém 44,54% após quantização INT8, o ponto mais eficiente do estudo em acurácia por megabyte entre modelos estáveis sob quantização. Mais notável ainda: a ablação da Fase 9 mostra que a maior parte do ganho de acurácia de uma arquitetura híbrida bem mais complexa (*stem* + *Fire* + residual, 49,79% FP32) vem de um único componente — o atalho residual — que pode ser adicionado ao AlexNetFire original sem nenhum parâmetro extra e sem prejudicar, inclusive melhorando, a robustez sob quantização INT8 (49,03% $\rightarrow$ 49,86%). Compensação estrutural leve, e especificamente atalhos residuais de custo zero, mostram-se mais decisivos para recuperar acurácia do que kernels maiores ou arquiteturas híbridas elaboradas.

Medições de hardware em duas GPUs (RTX 4060 Laptop e RTX 4090) confirmam que essas arquiteturas realmente acionam kernels Winograd em *batches* pequenos, mas que a vantagem depende de convoluções densas ($\text{groups}=1$) e compete com *tensor cores* em *batches* maiores — uma limitação da GPU como plataforma de validação, não do princípio de restrição de kernel, já que o acelerador FPGA que motiva este projeto não possui *tensor cores*.

Trabalhos futuros incluem: (i) *fine-tuning* do modelo podado estruturalmente na Fase 9 (AlexNetBottleneck com 46% menos parâmetros) para recuperar a acurácia hoje perdida pela poda sem re-treino, e generalização da poda além do bloco *bottleneck* (hoje restrita ao seu *mid_ch* interno) para cobrir também o módulo *Fire*; (ii) um *pipeline* de *weight-sharing QAT-aware* (com re-treino), já que o *clustering* pós-hoc avaliado na Fase 9 — apesar de tolerado bem pela família AlexNet — ainda perde para o *pipeline* de QAT $\rightarrow$ INT8 já em uso; (iii) extensão das medições de hardware (latência, potência) às famílias de arquitetura ainda não perfiladas; e (iv) arquiteturas eficientes baseadas em atenção (ViT híbrido), planejadas como Fase 8 mas ainda não implementadas.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [2] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

- [3] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," *arXiv preprint arXiv:1602.07360*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [6] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," Stanford CS231N Report, 2015.